

Small-World Communication Fabrics for Neuromorphic Multicore-SoCs

Sebastian Billaudelle*, Christian Metzner*, Jimmy Weber*, Zhe Su*, Chenxi Wen*, Siqu Liu*,
Laura Kriener*, Filippo Moro*, Giacomo Indiveri*, Melika Payvand*

*Institute of Neuroinformatics, University of Zurich and ETH Zurich, Zurich, Switzerland

Email: {sebastian, melika}@ini.uzh.ch

Abstract—As neuromorphic systems scale beyond a single core, inter-core event communication can become a dominant contributor to memory footprint, latency, and energy consumption. Biological neural systems address a similar scaling challenge through small-world organization, combining dense local connectivity with sparse long-range projections. In this work, we compare two recent multicore neuromorphic systems implemented in the same 22-nm FDSOI technology and explicitly optimized for such connectivity. The first, NeoCorAl, uses an asynchronous packet-switched tree with hierarchical multicast, whereas the second, MOSAIC, employs an RRAM-based, circuit-switched two-dimensional mesh that performs routing in memory. We examine the resulting trade-offs in routing flexibility, hop count, memory requirements, multicast efficiency, and scalability. We further study how the relative efficiency of tree- and mesh-based routing depends on communication locality in spatially-embedded, random, and layered networks. Finally, we discuss routing-aware training as a means of jointly optimizing neural connectivity, task performance, and hardware mappability.

Index Terms—routing, small-world connectivity, network-on-chip, neuromorphic systems, multi-core chips

I. INTRODUCTION

The energy cost of computing increasingly lies in moving data rather than performing computation [1]. Neuromorphic processors reduce this cost by distributing memory and computation across neural cores and communicating sparse events instead of dense numerical activations. However, the capacity of an individual core is limited by the size of its neuron and synapse arrays. Large neural networks must therefore be partitioned across multiple cores, and the cost of transmitting events between them can become a substantial fraction of the system area, energy, and latency [2].

Inter-core communication is often implemented using Address-Event Representation (AER) in which the identity of a firing neuron is encoded into a digital event and routed toward its destination synapses [3]. Supporting arbitrary neuron-to-neuron connectivity is highly flexible, but requires routing and interconnect resources. These costs grow with network size and fan-out, motivating architectures that restrict supported connectivity or exploit structured communication. Typically, neuromorphic systems group neurons into cores with dense local connectivity and use a more constrained fabric for the sparser communication between cores [2].

This organization is consistent with biological neural systems, where connectivity is predominantly local and decreases with physical distance, resulting in sparse projections across distant regions. The resulting small-world organization combines highly connected local neighborhoods with sparse global integration [4]. For electronics, the same structure is attractive for physical reasons, where local connections require shorter wires and fewer routing resources, whereas global communication incurs repeated switching, buffering, and memory-access costs. Locality can therefore be treated as an architectural principle for scalable neuromorphic systems [5].

Existing neuromorphic processors realize this principle with different communication topologies. *Tree-based and hierarchical fabrics* provide short paths across large systems and naturally support multicast by replicating events as they descend the hierarchy. Neurogrid, for example, connects chips through a binary multicast tree, while hierarchical AER schemes extend local buses across steadily larger distances [6-8]. Such fabrics offers efficient global reach, but shared links toward upper levels can limit bandwidth and lead to congestion.

Mesh-based fabrics forward events hop-by-hop through neighboring routers in a regular physical topology. TrueNorth [9] uses an asynchronous two-dimensional (2D) mesh with destination-based dimension-order routing, SpiNNaker [10] uses a 2D triangular torus with routing-table-based multicast, and Loihi [11] uses an asynchronous 2D mesh with dimension-order unicast routing and source-side replication for multicast. BrainScaleS-1 follows a more statically configured approach, establishing event paths through wafer-scale buses [12, 13]. Meshes provide a regular physical implementation, distributed bandwidth, and efficient communication between nearby cores, but the number of traversed links increases with source-destination distance.

Many systems combine these approaches. DYNAP-SE and DYNAP-SE2 both use hierarchical routing within a chip and a two-dimensional inter-chip mesh, together with local tag-based broadcast [14, 15]. BrainScaleS-1 takes the opposing approach of combining the wafer-wide mesh with tree-based inter-wafer routing. Earlier AER work also explored flat buses, broadcast grids, pre-structured interconnects, hierarchical networks, and router meshes, each offering a different trade-off between flexibility, bandwidth, latency, and routing-memory cost [16].

Despite their differences, these architectures share a motif: local communication within neural clusters is made inexpen-

sive, while expensive communication over larger distances is progressively more constrained. The central design question is therefore how flexible the global connectivity should be and how communication cost should scale with distance.

In this work, we study this question through two recent multi-core neuromorphic systems designed explicitly for locally dense and globally sparse networks. Both are implemented in 22 nm Fully-Depleted Silicon on Insulator (FDSOI) technology, allowing their architectural choices to be examined without conflating them with differences in the underlying technology. Nevertheless, they adopt fundamentally different communication fabrics: *NeoCorAL* uses an asynchronous packet-switched architecture with hardware support for tree-based multicast. Routing information is processed dynamically as events traverse the network, providing flexible one-to-many communication and short paths to distributed cores. *MOSAIC* arranges neural and routing tiles in a two-dimensional mesh and uses Resistive Random Access Memory (RRAM) crossbars to statically configure event paths. It extends in-memory computing to *in-memory routing*, replacing conventional routing Look-Up Tables (LUTs) with non-volatile circuit-switched connectivity [17]. We compare these approaches in terms of routing flexibility, hop count, memory footprint, and scalability. We then evaluate how their relative advantages depend on communication locality. Finally, we discuss how routing constraints can be incorporated during network optimization, rather than treated only as post-training mapping limitations.

II. ASYNCHRONOUS PACKET-SWITCHED TREE ROUTING

NeoCorAL is a mixed-signal multi-core neuromorphic processor whose inter-core spikes travel on an asynchronous, packet-switched tree Network-on-Chip (NoC) realizing the AER protocol. Spikes are arbitrated, encoded, routed through asynchronous switches, and decoded at the destination to drive target synapses. The fabric consists of four levels: a cortex-inspired *hierarchical routing model* for small-world connectivity, the *core interface* (output arbitration/encoding and input routing memory), the *data switch* (transport), and the *addressing scheme* (routing).

A. Cortex-inspired hierarchical routing

The NeoCorAL NoC follows cortical small-world connectivity, where long-range links become less frequent with distance [18]. Restricting the supported Spiking Neural Networks (SNNs) to this class shrinks the routing memory. NeoCorAL maps physical distance onto a router hierarchy: R0 broadcasts within a densely connected core (no intra-core specificity), R1 links n local cores, and R2 links n R1 routers. Higher routing levels expose progressively fewer connections, reducing routing-memory requirements. Each router computes distance from local logic and memory and updates it in the packet as it ascends the tree, bounding the per-hop address space. A hardware-aware placement algorithm maps SNNs by extracting densely connected cliques into cores and allocating the numerous short-range connections before the sparse long-range ones. Since all cores at a level share a fixed address

width, memory grows slowly with fan-in: a one-million-neuron network needs ~ 67 Mbit, about $98\times$ less than DYNAP-SE [14] and $307\times$ less than TrueNorth [19].

B. Core interface: arbitration and CAM routing memory

On the output side, a *Hierarchical Arbiter Tree* (HAT) time-multiplexes parallel neuron outputs onto the NoC [20]. Four-input arbiters progressively merge spikes with low latency. On the input path, an asynchronous Content Addressable Memory (CAM) routing LUT filters events by single-cycle parallel search which is built on a NOR-type cell with current-race match-line sense amplifier.

C. Asynchronous data-switch micro-architecture

The transport layer uses an ultra-low-cost asynchronous switch designed for high-fan-out spiking traffic [21]. It uses a two-phase bundled-data protocol requiring one control transition per transfer, enabling high throughput without a clock and with minimal idle activity. Its timing constraints are verified automatically using a commercial CAD flow.

Spikes are transmitted as compact *single-flit packets*, with the header and payload combined into a single word. In each $N \times N$ switch, an Input Port Module (IPM) buffers the packet, reads its header, and sends a request to each selected output. Each Output Port Module (OPM) arbitrates competing requests, forwards the selected packet through the crossbar, and stores it in an output Mousetrap buffer. For multicast, the crossbar replicates the packet to all selected OPMs, which can receive and forward it independently.

D. Multicast addressing encoding scheme

The routing layer encodes target cores within the single-flit header while reusing the switch microarchitecture [22]. NeoCorAL's *Hierarchical Bit String* (HBS) scheme instead uses a k -bit mask per level to select the sub-regions that compose each multicast tree, so routing bits scale as $k/\log_2 k$ and addressing capability as $(2^k - 1)^{1/\log_2 k}$. HBS maps directly onto the switch tree through a *relative address with bit rotation* (MSB = local branch), so every switch at a level shares identical, purely combinational routing logic.

III. RRAM-BASED CIRCUIT-SWITCHED MESH ROUTING

The MOSAIC architecture is likewise optimized for small-world network topologies, but – despite all commonalities – differs fundamentally from the system described in Section II. MOSAIC relies heavily on embedded non-volatile memory (eNVM) elements for both neural computation and on-chip event routing. Going beyond traditional neuromorphic in-memory computing approaches, it extends the concept of in-memory computing to event routing. This choice of technology has strongly influenced the overall design, and this section discusses the resulting design decisions and trade-offs.

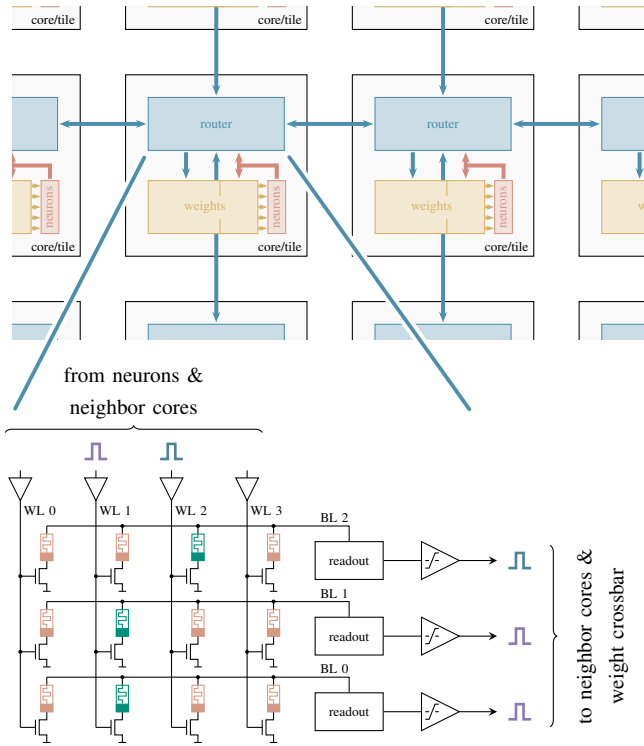


Fig. 1. The MOSAIC multi-core architecture. Dense local connectivity is augmented by a more constraint mesh-based routing for long-range connections. Both weight and routing crossbars are based on non-volatile memory devices, implementing in-memory computing and routing paradigms.

A. The MOSAIC architecture

MOSAIC is designed as a highly scalable architecture featuring a homogeneous mesh of neuromorphic cores (Figure 1). Each core comprises Leaky Integrate and Fire (LIF) neurons, which are implemented in pure Complementary Metal-Oxide-Semiconductor (CMOS) circuits, as well as a weight crossbar based on RRAM devices for weight storage and the computation of input activations. This crossbar is separated into two dedicated regions: a first block of synapses, each corresponding to two 1T1R branches to allow for positive as well as negative weights, realizes local all-to-all connectivity. The second region receives inputs delivered by the core’s router. In addition to these dense core-internal recurrent connections, neurons can therefore receive input from additional sources, typically corresponding to longer-range inter-core projections.

B. RRAM-based in-memory routing

Cores exchange events with their neighbors in all four cardinal directions, thereby forming a regular 2D mesh (Figure 1). Communication routes are programmed statically and are inherently non-volatile because they are stored in RRAM devices. Consequently, the system can be suspended without losing its network configuration and requires neither weight nor route reloading after power-up. The resulting communication scheme can therefore be characterized as non-volatile, statically configured circuit-switched routing. Routes may connect neurons in directly adjacent cores or bypass intermediate cores to realize longer-range projections. Moreover, routes can

branch, enabling individual events to be delivered to multiple destinations and thus supporting multicast communication.

The router itself is implemented as an RRAM-based switch matrix. The crossbar’s wordlines serve as inputs, and incoming events activate the corresponding wordlines and selected RRAM devices. A device programmed to the low-resistance state represents an active route, whereas a device in the high-resistance state represents an inactive connection. The crossbar’s bitlines represent the outgoing routes, and an outgoing event is triggered when increased conductance is detected on a bitline. This approach requires a careful balance among device characteristics, crossbar sizes, and event statistics to avoid spurious events and spike loss. Merging route memory with the switch implementation itself eliminates memory transfers and thus enables energy-efficient in-memory routing. Without additional flow control and tagging, however, the switch matrices do not support many-to-one projections, which could result in conflicts between coincident events. While the resulting one-hot encoding inherently decreases memory utilization and causes routing resources to scale linearly with the number of potential routes, this disadvantage is partly offset by the high density of the RRAM-based approach. Further improving memory utilization, MOSAIC employs buses with a 4 bit source-address representation, reducing the required crossbar size by a factor of 16.

The RRAM-based in-memory routing concept is compatible with both asynchronous and synchronous communication paradigms. The current implementation is fully synchronous, with each router hop introducing one clock cycle of latency. An asynchronous routing scheme is equally feasible and was, in fact, adopted in the first MOSAIC prototype [17]. However, RRAM crossbars and their readout circuitry are inherently analog and require well-defined pulses to ensure reliable read and programming operations. Consequently, asynchronous communication would still require local pulse-generation circuitry at each routing stage. The associated area and energy overhead would substantially reduce the latency and efficiency advantages typically attributed to asynchronous communication, making a synchronous implementation the more favorable trade-off in the present architecture.

IV. EFFICIENCY OF MESH- AND TREE-BASED ROUTING

NeoCorAL and MOSAIC were both designed around a small-world layout. Yet they adopt inherently different approaches for inter-core event routing, resulting in preferences for different network topologies. In the following, we compare binary- and quadtree as well as mesh routing fabrics (Figure 2A, bottom) in terms of routing cost across three network families: spatially embedded small-world networks, random networks, and feedforward layered networks. We measure routing cost in hops, defined as the number of edges that need to be traversed across the routing fabric reaching from a source to a destination core, and normalize this figure by the total number of connections. For each network and fabric, neurons are mapped onto cores to minimize routing cost [23].

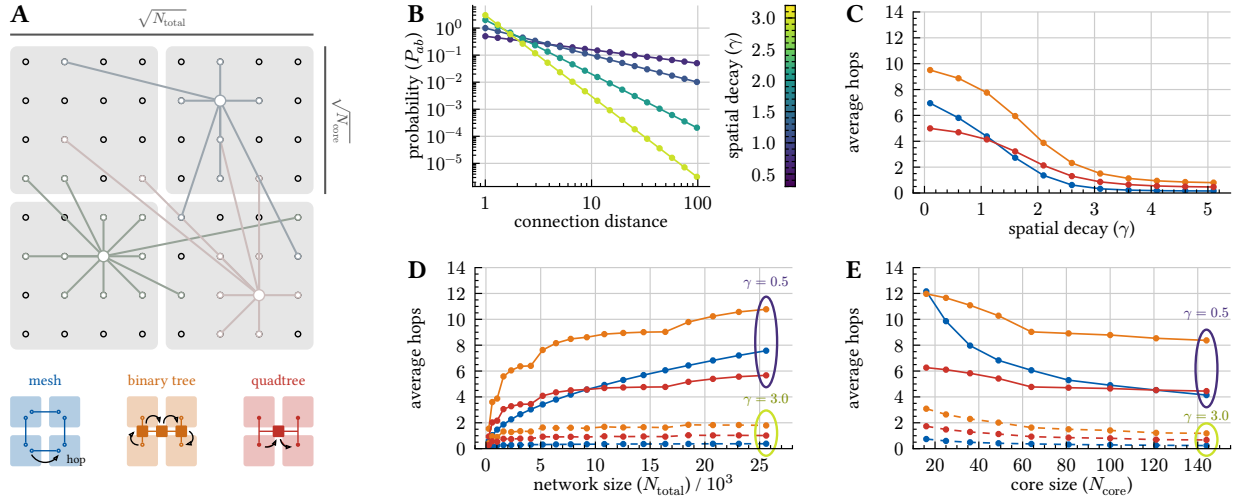


Fig. 2. Comparison of inter-core routing architectures on spatially-embedded small-world network graphs. **A** Illustration of a spatially-embedded small-world network on a 2D grid of neurons. The connectivity of three units is exemplarily highlighted. This logical network can be mapped to a virtual multi-core system by partitioning the 2D neuronal grid into cores. These cores can be outfitted with varying routing schemes, e.g., a mesh or tree-based architectures. **B** A Kleinberg network’s connectivity profile can be tuned via the spatial decay parameter γ . **C** A router’s efficiency, here represented by the average number of hops per connection, depends especially on the density of long-range connectivity. Mesh routing excels at more local networks (high γ), while quadtree architectures are better suited for networks with a higher degree of global connectivity (low γ). **D** High- γ networks scale well with growing size N_{total} and allow mesh fabrics to exploit their efficient one-hop routing across neighboring cores. Quadtree routing becomes more efficient for Globally dense networks (low γ) with increasing connection distances. In that case, **E** Mesh routing profits from larger cores, effectively reducing the distance (in hops), while a quadtree topology can cope with smaller cores more efficiently. Average fanout per neuron is fixed to 8 across all sweeps.

A. Spatially embedded small-world networks

Spatially-embedded *navigable* small-world connectivity is characterized by a connection probability that decays as a power law with the physical distance between nodes [24].

We compare the average number of routing hops required to navigate such spatially-embedded small-world graphs mapped onto a virtual multi-core system (Figure 2A). Following [24], each neuron of the system is assigned a location on a 2D grid and connected to its nearest neighbors. A fixed number of long-range fan-out connections are generated per neuron with probability $P_{ab} \propto d_{ab}^{-\gamma}$ where P_{ab} is the probability of connecting neurons a and b and d_{ab} is the Euclidean distance between them on the 2D spatial embedding grid. The clustering exponent γ controls the trade-off between spatial locality and long-range communication across the small-world network, with higher γ favoring more local connections while lower γ leads to spatially distributed networks (Figure 2B).

Figure 2C indicates that the average number of routing hops required to navigate a mapped small-world network decreases monotonically with the clustering exponent γ for all three routing architectures, provided a fixed number of total neurons ($N_{\text{total}} \approx 16,000$) and neurons per core ($N_{\text{core}} = 64$), as well as a fixed average fanout of 8. In general, the binary tree requires more average routing hops than both mesh and quadtree. At low γ , local connections dominate the network and the mesh benefits from the fact that neighboring cores are only one hop away while trees may need to navigate several levels of the hierarchy to connect neurons in cores that are spatially close but on different subtrees. However, low γ networks that are distributed widely across the 2D grid favor the quadtree.

In Figure 2D, we observe that tree routing scales more favorably than a mesh when increasing the network size N_{total}

(N_{core} fixed to 64, average fanout of 8) – as long as γ is sufficiently low. However, increasing the number of neurons per core reduces the mesh routing cost even for a low γ by decreasing the number of inter-core hops, as shown in Figure 2E where N_{core} is swept while fixing $N_{\text{total}} \approx 16,000$. This indicates that the range of spatially-embedded networks for which tree routing is advantageous decreases as more neurons are mapped to each core.

Overall, we find that mesh routing architectures shine on highly localized networks, while tree routing is best suited for larger networks that are more widely distributed across cores, especially in the case of relatively small cores.

B. Random networks

In contrast to small-world networks, fully random populations lack any spatial structure and their connectivity is often instead governed by a globally uniform probability density. We thus consider them as a benchmark for how mesh and tree routing behave once spatial locality can no longer be exploited.

Figure 3A shows the average routing hops versus network size (N_{total}) at fixed average fan-out and $N_{\text{core}} = 64$. For small networks, mesh routing benefits from the fact that most inter-core connections are between adjacent cores and outperforms both tree routers. The quadtree becomes advantageous at larger network sizes, mirroring the low- γ regime identified in Figure 2D. Figure 3B shows routing cost as a function of average fan-out, while keeping network size and thus the number of cores constant. Here, quadtree scales most favorably.

Overall, random connectivity prevents mesh routing from profiting from strong locality, and does so more severely as the network grows: for small or sparse random networks all

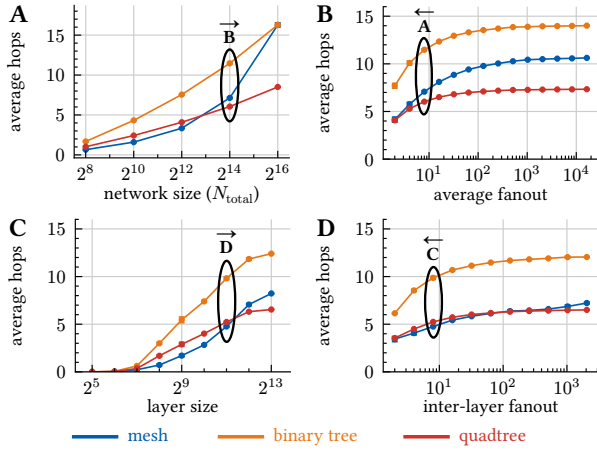


Fig. 3. Comparison of inter-core routing cost for random (A, B) and layered feedforward networks (C, D), with $N_{core} = 64$ and $N_{total} = 16,384$ unless specified otherwise. Panels A and B as well as C and D represent two cuts through 2D parameter spaces; the fixed parameter is marked accordingly in the partner graph. **A** In random networks, the average number of hops per connection increases with growing network size N_{total} (average fan-out fixed at 8). Mesh routing becomes increasingly inefficient for larger networks. **B** Quadtree routing excels with an increasing fan-out and an increasing number of long-range connections. **C** Mesh routing maintains an advantage on feedforward networks with small- to medium-sized layers (inter-layer sparsity fixed at 2^{-8}). **D** Mesh and quadtree routing fabrics exhibit a similar efficiency, independent of the inter-layer sparsity (8 layers of size 2048).

three fabrics are comparable, but for large or dense ones, the quadtree consistently offers lower routing costs.

C. Feedforward layered networks

Especially in machine learning applications, networks are often highly structured and directed. Feedforward networks feature dense projections between consecutive layers but disregard layer-internal recurrence. In contrast to small-world or fully random networks, placement algorithms can exploit the directionality for optimized routing.

In Figure 3C, we consider a network of constant size and inter-layer sparsity. For small layer size, connections remain mostly local and typically span only neighboring cores. With growing layer sizes, layers begin to span multiple cores and the resulting placement can no longer fully exploit the directional data flow, thus incurring longer routes and a higher routing cost. In this case, tree routers become more efficient, while mesh fabrics are optimal as long as communication remains mostly local. In Figure 3D, the number of layers is fixed at 8, resulting in layers of size 2048, and the average inter-layer fan-out is swept. Mesh and quadtree fabrics perform very similarly and remain more efficient than binary tree routing – unlike in the random-network case (Figure 3B) where the quadtree architecture is the best performing at high fan-out.

Preserving the structure and directionality of layered networks when mapping to a multi-core system is crucial to keep connectivity local. Only then does mesh routing fully exploit the efficiency of communicating between adjacent cores.

V. ROUTING-AWARE NEURAL NETWORK TRAINING

As shown in Section IV, increasing spatial locality in small-world connected networks reduces the average number of rout-

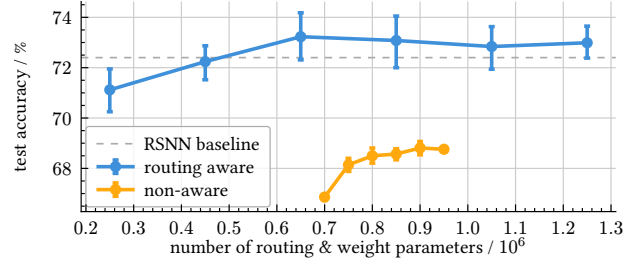


Fig. 4. Test accuracy on the SHD data set as a function of the routing and weight memory count in the MOSAIC architecture [17], trained with and without the routing-awareness, compared against the original vanilla Recurrent Spiking Neural Network (RSNN) results [27]. Adapted from [25].

ing hops required by the networks considered here. However, these gains are only realized if the neural network can be placed and routed efficiently on the hardware. The required resources depend not only on the number of connections, but also on where neurons are placed and how far their connections reach. A network may therefore satisfy the overall parameter budget yet use the routing resources inefficiently, or fail to map altogether because the connectivity supported by particular neuron or routing tiles is exceeded. Constraint-aware network optimization is thus central to exploiting the efficiency of a small-world communication fabric. Common approaches to this problem face significant trade-offs: mapping only topologies that fit the hardware restrict adaptability, evolutionary architecture searches incur high computational costs, and gradient-based methods struggle with discrete, non-differentiable mapping and routing operations.

The routing-aware method in [25] adapts DeepR [26] to jointly optimize task performance and hardware mappability for MOSAIC. During training, an ℓ_1 regularizer drives weak weights to zero, after which low-magnitude connections are pruned and reassigned while preserving a hardware-compatible distribution of connections across routing distances. This allows the network topology to adapt to the task without exceeding neuron- and routing-tile resources. Avoiding routing at every training step, the distance-dependent connectivity profile is used as an efficient proxy for mappability.

As shown in Figure 4, applying this method to the Spiking Heidelberg Digits (SHD) task produces networks that are directly mappable onto MOSAIC. Compared with training a fixed hardware-compatible connectivity, routing-aware optimization achieves approximately five percentage points higher accuracy for the same memory budget and requires about one order of magnitude less memory at iso-accuracy [25].

VI. DISCUSSION

Our comparison shows that small-world connectivity does not imply a single optimal routing topology. Mesh-based fabrics efficiently exploit strongly local traffic, whereas hierarchical trees provide shorter routes when communication is less local or spans larger systems. The appropriate fabric therefore depends on the spatial statistics of the target networks.

At the hardware level, the central challenge is to retain sufficient routing flexibility without recreating an expensive global communication fabric. Packet-switched trees provide

flexible multicast and efficient global reach, but typically require routing tables and arbitration, and may experience congestion at higher levels of the hierarchy. In-memory circuit-switched meshes remove memory transfer and favor regular, local communication, but impose stronger constraints on route configuration and supported traffic.

These constraints also create an algorithmic challenge. The benefits of a locality-optimized fabric are only realized when neural connectivity, placement, and routing are optimized together. As demonstrated by routing-aware training, two networks with the same number of parameters can require substantially different routing resources depending on how their connections are distributed in space. Compilation and learning should therefore be treated as part of the architectural design process. More broadly, locality can serve as an inductive bias toward modular computation, in which locally connected subnetworks specialize and communicate through a smaller number of structured long-range links.

Three-dimensional integration could further reshape these trade-offs. Back-end of Line (BEOL) memory integrated above logic can store synaptic and routing state close to computation, while sequential 3D integration can provide dense vertical connections between nearby neural and routing layers. At larger scales, hybrid bonding and through-silicon vias could connect clusters through a smaller number of long-range links [28]. This would naturally match small-world connectivity: dense, inexpensive communication within local three-dimensional neighborhoods and sparse communication between them. However, vertical interconnects, thermal budgets, and heterogeneous integration remain important constraints, making hardware-algorithm co-design essential.

In conclusion, communication efficiency in multi-core neuromorphic systems is not determined by routing topology alone. It emerges from the interaction between neural connectivity, physical communication fabric, placement and routing, and network optimization. Small-world organizations provides a common principle for this co-design, while mesh, tree, and future 3D fabrics offer complementary ways of realizing it.

REFERENCES

- [1] Mark Horowitz. 1.1 computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 10–14, 2014.
- [2] Aaron R Young, Mark E Dean, James S Plank, and Garrett S Rose. A review of spiking neuromorphic hardware communication systems. *IEEE Access*, 7:135606–135620, 2019.
- [3] Kwabena A Boahen. Communicating neuronal ensembles between neuromorphic chips. In *Neuromorphic Systems Engineering: Neural Networks in Silicon*, pages 229–259. Springer, 1998.
- [4] Ed Bullmore and Olaf Sporns. Complex brain networks: graph theoretical analysis of structural and functional systems. *Nature reviews neuroscience*, 10(3):186–198, 2009.
- [5] Dhireesha Kudithipudi, Catherine Schuman, Craig M Vineyard, Tej Pandit, Cory Merkel, Rajkumar Kubendran, James B Aimone, Garrick Orchard, Christian Mayr, Ryad Benosman, et al. Neuromorphic computing at scale. *Nature*, 637(8047):801–812, 2025.
- [6] Ben V Benjamin, Gao Peiran, Emmett McQuinn, and et al. Neurogrid: A mixed-analog-digital multichip system for large-scale neural simulations. *Proceedings of the IEEE*, 102(5):699–716, 2014.
- [7] Paul Merolla, John Arthur, Rodrigo Alvarez-Icaza, Jean-Marie Bussat, and Kwabena Boahen. A multicast tree router for multichip neuromorphic systems. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 61(3):820–833, 2014.
- [8] Jongkil Park, Theodore Yu, Siddharth Joshi, Christoph Maier, and Gert Cauwenberghs. Hierarchical address event routing for reconfigurable large-scale neuromorphic systems. *IEEE transactions on neural networks and learning systems*, 28(10):2408–2422, 2016.
- [9] Paul A Merolla, John V Arthur, Rodrigo Alvarez-Icaza, and et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197):668–673, 2014.
- [10] Steve B Furber, Francesco Galluppi, Steve Temple, and Luis A Plana. The spinnaker project. *Proceedings of the IEEE*, 102(5):652–665, 2014.
- [11] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, et al. Loihi: A neuromorphic manycore processor with on-chip learning. *Ieee Micro*, 38(1):82–99, 2018.
- [12] Johannes Schemmel, Daniel Brüderle, Andreas Grübl, Matthias Hock, Karlheinz Meier, and Sebastian Millner. A wafer-scale neuromorphic hardware system for large-scale neural modeling. In *2010 IEEE international symposium on circuits and systems (iscas)*, pages 1947–1950. IEEE, 2010.
- [13] Hartmut Schmidt, José Montes, Andreas Grübl, Maurice Güttler, Dan Husmann, Joscha Ilmberger, Jakob Kaiser, Christian Mauch, Eric Müller, Lars Sterzenbach, Johannes Schemmel, and Sebastian Schmitt. From clean room to machine room: commissioning of the first-generation BrainScaleS wafer-scale neuromorphic system. *Neuromorphic Computing and Engineering*, 3(3):034013, 2023.
- [14] Saman Moradi, Ning Qiao, Fabio Stefanini, and Giacomo Indiveri. A 28 nm fd-soi neuromorphic processor with asynchronous synapse dynamics and online learning. *IEEE Transactions on Biomedical Circuits and Systems*, 12(1):106–122, 2018.
- [15] Ole Richter, Chenxi Wu, Adrian Whatley, German Köstinger, Carsten Nielsen, Ning Qiao, and Giacomo Indiveri. DYNAP-SE2: a scalable multi-core dynamic neuromorphic asynchronous spiking neural network processor. *Neuromorphic Computing and Engineering*, 4(1):014003, 2024.
- [16] Carlos Zamarreño-Ramos, Alejandro Linares-Barranco, Teresa Serrano-Gotarredona, and Bernabé Linares-Barranco. Multicasting mesh aer: A scalable assembly approach for reconfigurable neuromorphic structured aer systems. application to convnets. *IEEE transactions on biomedical circuits and systems*, 7(1):82–102, 2012.
- [17] Thomas Dalgaty, Filippo Moro, Yigit Demirağ, Alessio De Pra, Giacomo Indiveri, Elisa Vianello, and Melika Payvand. Mosaic: in-memory computing and routing for small-world spike-based neuromorphic systems. *Nature Communications*, 15(1):142, 2024.
- [18] Vanessa Leite, Zhe Su, Adrian Whatley, and Giacomo Indiveri. Cortical-inspired placement and routing: minimizing the memory resources in multi-core neuromorphic processors. In *2022 IEEE Biomedical Circuits and Systems Conference (BioCAS)*, pages 364–368, 2022.
- [19] Filipp Akopyan, Jun Sawada, Andrew Cassidy, Rodrigo Alvarez-Icaza, John Arthur, Paul Merolla, Nabil Imam, Yutaka Nakamura, Pallab Datta, Gi-Joon Nam, Brian Taba, Michael Beakes, Bernard Brezzo, Jente Kuang, Rajit Manohar, William Risk, Bryan Jackson, and Dharmendra Modha. TrueNorth: Design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 34(10):1537–1557, 2015.
- [20] Zhe Su, Hyunjung Hwang, Tristan Torchet, and Giacomo Indiveri. Core interface optimization for multi-core neuromorphic processors. In *2023 28th IEEE International Symposium on Asynchronous Circuits and Systems (ASYNC)*, pages 89–98, 2023.
- [21] Zhe Su, Simone Ramini, Demetra Coffen Marcolin, Alessandro Veronesi, Milos Krstic, Giacomo Indiveri, Davide Bertozzi, and Steven M. Nowick. An ultra-low cost and multicast-enabled asynchronous noc for neuromorphic edge computing. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 14(3):409–424, 2024.
- [22] Zhe Su, Aron Bencsik, Giacomo Indiveri, and Davide Bertozzi. An efficient multicast addressing encoding scheme for multi-core neuromorphic processors. In *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5, 2025.
- [23] Jimmy Weber, Chenxi Wu, and Melika Payvand. Gmap: An open-source efficient compiler for mapping any network onto any neuromorphic chip.

In *Proceedings of the 2023 International Conference on Neuromorphic Systems*, 2023.

- [24] Jon M. Kleinberg. Navigation in a small world. *Nature*, 406(6798):845–845, August 2000.
- [25] Jimmy Weber, Theo Ballet, and Melika Payvand. Hardware architecture and routing-aware training for optimal memory usage: a case study. In *2025 Neuro Inspired Computational Elements (NICE)*, pages 1–5. IEEE, 2025.
- [26] Guillaume Bellec, David Kappel, Wolfgang Maass, and Robert Legenstein. Deep rewiring: Training very sparse deep networks. *arXiv preprint arXiv:1711.05136*, 2017.
- [27] Benjamin Cramer, Yannik Stradmann, Johannes Schemmel, and Friedemann Zenke. The heidelberg spiking data sets for the systematic evaluation of spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 2020.
- [28] Elisa Vianello and Melika Payvand. Scaling neuromorphic systems with 3D technologies. *Nature Electronics*, 7(6):419–421, 2024.